

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A Project Proposal
on
“SpotMe”

[Code No.: COMP 308]

(For partial fulfillment of Year III / Semester II in Computer Engineering)

Submitted by
Sijan Bhandari (9)
Ayush Paudel (31)
Bigya Poudel (36)
Lawan Poudyal (37)
Abhiyan Regmi (47)

Submitted to
Project Coordinator
Department of Computer Science and Engineering

April 27, 2026

Abstract

This project presents the development of SpotMe, a cloud-native web application designed to streamline large-scale event photo management and retrieval using facial recognition and vector similarity search. Built with React for the frontend and an Express.js backend for core application logic and Application Programming Interface (API) handling, the system integrates a FastAPI-based microservice dedicated to high-performance facial processing tasks, including face detection, embedding generation, and clustering. Secure authentication is managed using Better Auth, while persistent data storage is handled through Supabase-hosted PostgreSQL with Prisma Object Relational Mapping (ORM), ensuring scalability and efficient data access. The platform enables event organizers to upload extensive image collections into isolated event spaces, triggering an automated background pipeline that preprocesses images, extracts high-dimensional feature representations, and organizes them into identity-based clusters indexed for fast similarity lookup. By offloading computationally intensive operations to the FastAPI microservice during the ingestion phase and leveraging optimized indexing strategies, the system minimizes runtime overhead and achieves low-latency retrieval performance. The overall architecture emphasizes modularity, separation of concerns, and efficient resource utilization, delivering a robust, scalable, and production-ready solution for managing and retrieving event-based visual data.

Keywords: *Facial Recognition, Vector Similarity Search, Face Clustering, Event Photo Management, Cloud-Native, React, FastAPI, Microservice, Face Embedding, Image Retrieval*

Contents

1	Introduction	1
1.1	Background	1
1.2	Objectives	2
1.3	Motivation and Significance	2
1.4	Expected Outcomes	3
2	Related Works	4
2.1	Google Photos	4
2.2	Amazon Rekognition	4
3	Procedure and Methods	6
4	System Requirement Specifications	10
4.0.1	Developer Requirements	10
4.0.2	User Requirements	10
5	Project Planning and Scheduling	11

List of Figures

2.1	Google Photos	4
2.2	Amazon Rekognition	5
3.1	System Architecture Diagram	7
3.2	Flowchart	9
5.1	Gantt chart	11

Abbreviations

AI Artificial Intelligence. 3

API Application Programming Interface. i, 6

GB Gigabyte. 10

Mbps Megabits per second. 10

ORM Object Relational Mapping. i, 2, 6, 7

RAM Random Access Memory. 10

URL Uniform Resource Locator. 7

Chapter 1: Introduction

This project presents SpotMe, a cloud-native web application designed to simplify large-scale event photo management and retrieval. It addresses the challenge of finding individual photos in massive image collections by using facial recognition and vector similarity search. Built with React, Express.js, and a FastAPI microservice for facial processing, the system enables efficient image handling, secure user access, and fast, personalized photo retrieval through a scalable and modular architecture.

1.1 Background

With the rapid growth of digital media and widespread smartphone usage, modern events generate massive volumes of visual data, making image management and retrieval increasingly difficult. Traditional methods that rely on manual browsing or basic metadata such as timestamps and filenames are inefficient, time-consuming, and fail to scale for large datasets. Recent advancements in machine learning, particularly in computer vision, have enabled automated image analysis through facial recognition and feature extraction. By representing faces as high-dimensional vector embeddings, systems can perform similarity-based searches, allowing fast and accurate retrieval of visually similar images and shifting from simple keyword-based approaches to intelligent, content-based retrieval.

At the same time, modern software development emphasizes scalability, modularity, and efficient resource utilization through microservices-based architectures. In this approach, system components operate independently and communicate through well-defined interfaces, improving maintainability and scalability. This project adopts a hybrid architecture where the main web application manages user interaction and data handling, while a FastAPI microservice performs computationally intensive facial processing. Asynchronous job processing using BullMQ ensures non-blocking handling of large image uploads. Despite existing cloud storage and image-sharing platforms, efficient personalized retrieval systems remain limited; therefore, SpotMe integrates modern web technologies, distributed processing, and machine learning to provide a scalable and practical solution for event photo management.

1.2 Objectives

- i. To develop a full-featured web application using React and Express.js that allows event organizers to create/manage events, upload large-scale image collections, and provide secure, user-specific access using Better Auth.
- ii. To design and implement a scalable PostgreSQL database (via Supabase and Prisma ORM) to handle users, events, images, metadata, and high-dimensional vector embeddings with efficient storage and retrieval.
- iii. To integrate BullMQ for asynchronous job processing and build a FastAPI-based microservice for facial recognition (face detection, embedding generation, clustering), ensuring efficient background processing and high-performance computation.
- iv. To implement vector similarity search for fast and accurate image retrieval, while ensuring overall system scalability, modularity, and efficient resource utilization through a well-structured, service-oriented architecture.

1.3 Motivation and Significance

We choose this project because managing and retrieving photos from large-scale events remains inefficient in existing systems. In events such as college programs, conferences, and social gatherings, thousands of images are captured and later shared without proper organization. Users are often required to manually browse through large collections to find their own photos, which is time-consuming and frustrating. Existing solutions lack intelligent retrieval mechanisms and fail to provide personalized access, making them unsuitable for handling large volumes of visual data.

SpotMe addresses these challenges by introducing an automated and intelligent workflow for both image processing and retrieval. Users can create or join event spaces, upload images, and initiate a background processing pipeline that handles large-scale data efficiently. The system processes uploaded images asynchronously, performing face detection and generating embeddings through a dedicated service. For retrieval, users can upload a query image, which is converted into an embedding and matched using vector similarity search to quickly identify and fetch relevant results. This approach ensures efficient handling of large datasets while maintaining fast response times.

The significance of this project lies in its ability to integrate modern web technologies with Artificial Intelligence (AI)-driven solutions to solve a real-world problem. By adopting a modular architecture, asynchronous processing, and scalable data management techniques, the system achieves high performance and reliability. It reduces manual effort for both users and event organizers, enhances accessibility to personalized content, and demonstrates the practical application of advanced concepts such as microservices, embeddings, and vector-based search in building scalable and efficient web applications.

1.4 Expected Outcomes

The successful completion of this project is expected to achieve the following outcomes:

- i. A complete, production-ready web application built with React and Express.js, featuring secure authentication via Better Auth, enabling event creation, large-scale image uploads, and personalized access to photo collections.
- ii. A scalable PostgreSQL database hosted on Supabase and managed via Prisma ORM, designed to handle complex relational data alongside high-dimensional vector embeddings for efficient storage and retrieval.
- iii. An asynchronous image processing pipeline using BullMQ integrated with a dedicated FastAPI microservice, handling face detection and embedding generation in a non-blocking, high-performance manner.
- iv. A low-latency vector similarity search system enabling fast and precise retrieval of relevant images based on facial embeddings, supported by a modular and scalable architecture ready for real-world deployment.

Chapter 2: Related Works

2.1 Google Photos

Google (2026) is a cloud-based photo storage and management platform that provides advanced features for organizing and searching images. It uses facial recognition technology to automatically detect and group faces across large photo collections, allowing users to search images by individuals without manual tagging. The system generates feature embeddings for detected faces and applies clustering techniques to group similar faces together.

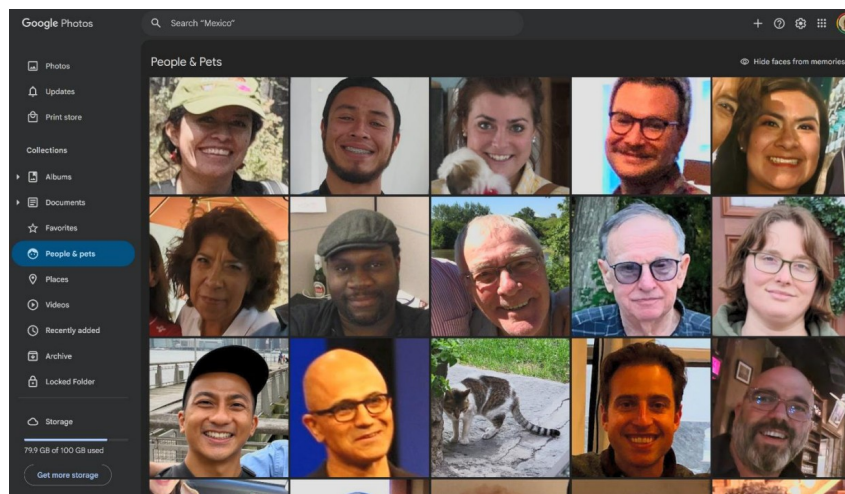


Figure 2.1: Google Photos

2.2 Amazon Rekognition

Amazon Web Services (2026) is a cloud-based image and video analysis service that provides capabilities such as face detection, face comparison, and face indexing using pre-trained machine learning models. It allows developers to analyze large collections of images and perform similarity-based searches by comparing facial features. The system supports scalable processing and efficient retrieval of matching faces from large datasets.

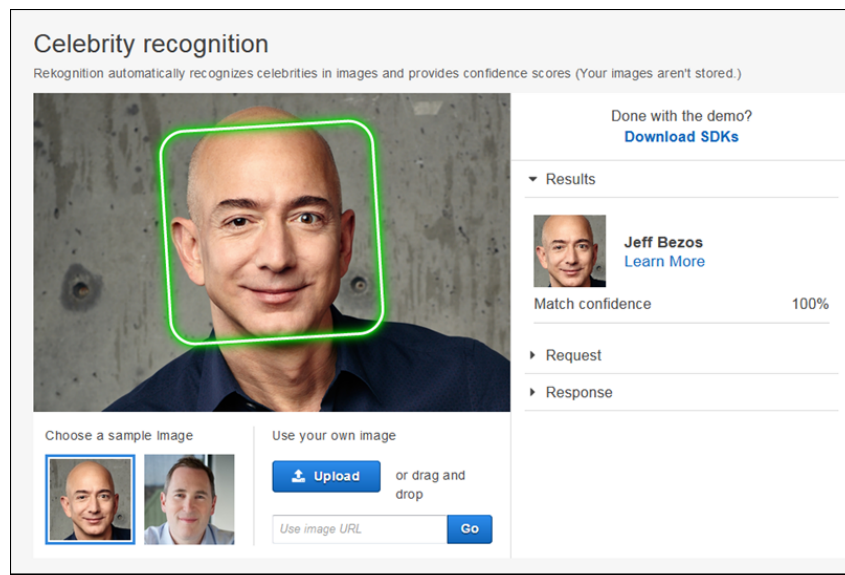


Figure 2.2: Amazon Rekognition

Chapter 3: Procedure and Methods

To build an efficient and scalable web-based platform for event photo retrieval, the system follows a structured and modular development process encompassing authentication workflow design, backend service orchestration, database and vector storage management, asynchronous processing pipelines, and frontend interface implementation. The primary goal is to provide event organizers and attendees with seamless access to large-scale image collections, enabling efficient organization, processing, and retrieval of photos through a unified and user-centric web application experience.

1. Requirement Analysis and Planning

We begin by defining the core objectives of the SpotMe web application: to enable secure user authentication, support event creation and management, allow large-scale image uploads, and provide efficient retrieval of event photos through facial recognition and vector similarity search. The system is designed to handle high volumes of image data while ensuring fast query performance and a seamless user experience. We also identify the required technologies: React for the frontend interface, Express.js for the core backend services, FastAPI as a microservice for computationally intensive facial processing tasks, BullMQ for asynchronous job handling, Supabase-hosted PostgreSQL managed via Prisma ORM for data storage, Cloudinary for image hosting, and Better Auth for secure authentication.

2. System Architecture and Design

As shown in Figure 3.1, the layers in the architecture are:

- **Frontend Layer:** Developed using React, responsible for rendering user interfaces such as authentication flows, event creation dashboards, image upload interfaces, and event-based photo galleries, ensuring a responsive and user-friendly experience.
- **Backend Layer:** Implemented using Express.js, which manages API endpoints, handles authentication via Better Auth, coordinates event and image management, and orchestrates communication between services including job queues and processing microservices.

- **Processing Layer:** A FastAPI-based microservice is responsible for computationally intensive tasks such as face detection, embedding generation, and clustering, operating independently to ensure high-performance processing.
- **Queue Layer:** BullMQ is used to manage asynchronous job processing, decoupling image upload operations from heavy background tasks and ensuring reliability and scalability under high workloads.
- **Database Layer:** Supabase-hosted PostgreSQL, managed via Prisma ORM, stores user data, event metadata, image references, and associated vector embeddings, enabling efficient querying and structured data management.
- **Storage Layer:** Cloudinary is utilized for scalable image storage and delivery, with uploaded images referenced via Uniform Resource Locator (URL)s within the system for further processing and retrieval.

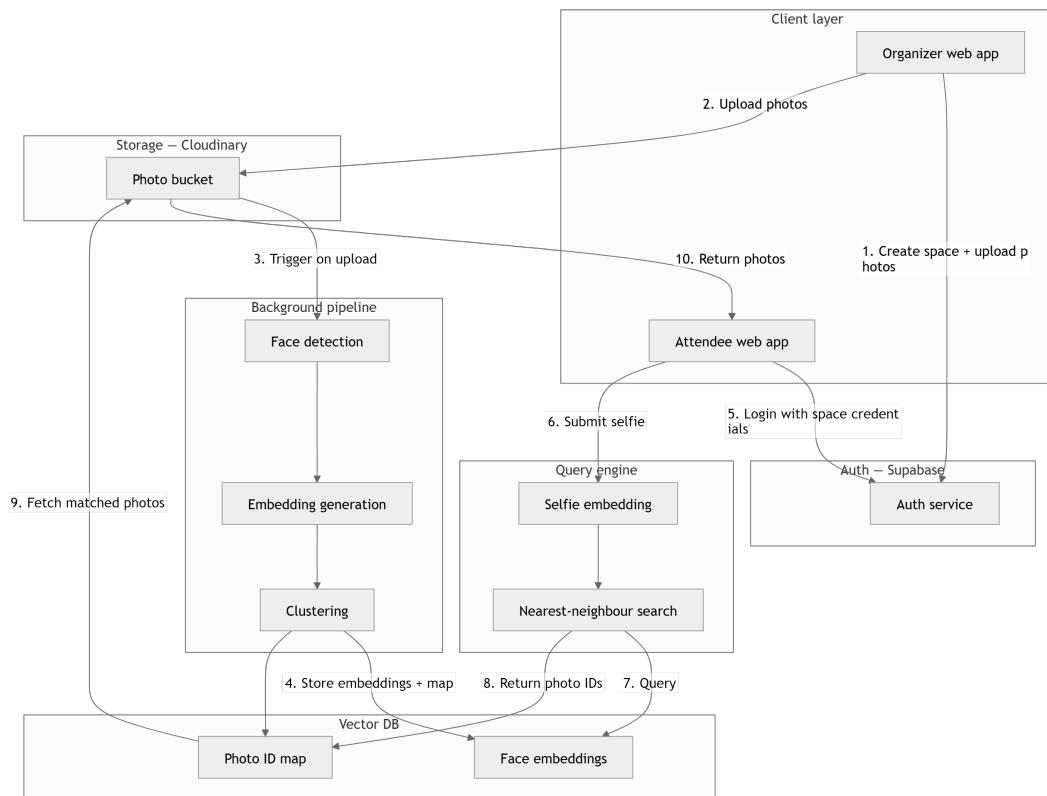


Figure 3.1: System Architecture Diagram

3. Feature Development and Integration

- **Authentication and Account Management:** Better Auth and backend endpoints manage secure user authentication, including login, session handling,

and access control to ensure that only authorized users can access event-specific data.

- **Event Management and Attendance:** Users can create and manage events, providing a dedicated space to organize and store event-related images.
- **Image Upload and Organization:** Users can upload images to their events, building a centralized and structured collection of event photos.
- **Event Sharing and Access Control:** Event owners can share access with others, allowing participants to join and interact with the event space.
- **Collaborative Participation:** Users can join events created by others and contribute by uploading their own images to the shared collection.
- **Personalized Photo Discovery:** Users can quickly locate photos of themselves within large event collections through an intuitive and user-centric retrieval experience.

4. Flowchart

As shown in Figure 3.2, the diagram illustrates the step-by-step sequence of how a user interacts with the system. It begins when a user visits the application and checks whether they are logged in; if not, they proceed through authentication and return to the main flow. Once on the dashboard, the user can either create an event or join one using a code, with validation ensuring only authorized access. Within the event space, the flow branches based on user actions: if images are uploaded, they are processed through a backend pipeline where they are stored, analyzed for face detection, converted into embeddings, and saved in the database before returning the user to the event. Alternatively, if the user searches for photos, a query image is uploaded, converted into an embedding, and compared against stored data using similarity search, after which matching images are retrieved and displayed. If no action is taken, the user remains within the event dashboard.

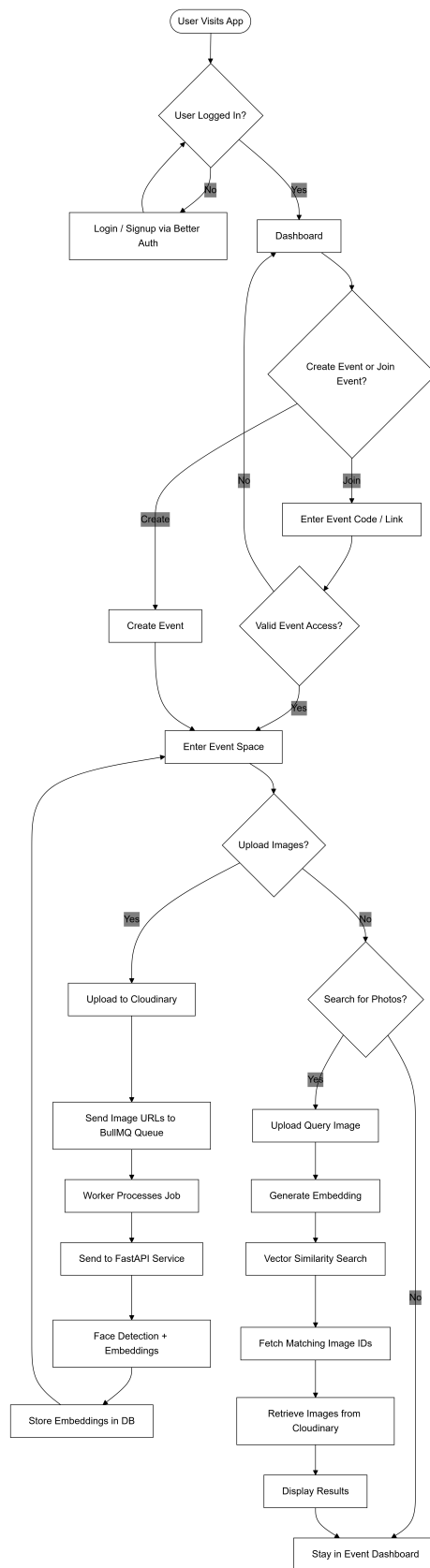


Figure 3.2: Flowchart

Chapter 4: System Requirement Specifications

The development of SpotMe requires both software and hardware support on the developer and user sides. These specifications ensure smooth development, deployment and usage of the system across different environments.

4.0.1 Developer Requirements

The developer environment consists of tools and platforms required to build, test, and maintain the SpotMe platform. These include software for frontend and backend development, database management, and machine learning model implementation.

- Operating System: Windows 10/11, macOS, or any Linux distribution
- Frontend: React.js (v19.2.5), Tailwind CSS (v4.2.4), TypeScript (v6.0.3)
- Backend: Node.js (v24.15.0), Express.js (v5.2.1), Python (v3.14.4)
- Database: PostgreSQL (v18.3)
- Machine Learning Tools: Scikit-learn (v1.8.0), NumPy (v2.4.4),

4.0.2 User Requirements

The user environment specifies the minimum hardware and software configuration needed to access and use SpotMe efficiently through a web browser without requiring any additional installations.

- Hardware: Any modern system with Intel Core i3 or better, 4 Gigabyte (GB) Random Access Memory (RAM), and integrated graphics
- Software: A recent version of Chrome, Firefox, Edge, or Safari on Windows, macOS, or Linux
- Connectivity: Stable internet connection (minimum 5 Megabits per second (Mbps) recommended)

Chapter 5: Project Planning and Scheduling

We are expecting to execute the project within 12 weeks by equally distributing the tasks to each group member. We all will be working continuously for successful implementation of the project. The following Gantt chart shows the time allocated for different branches of our project



Figure 5.1: Gantt chart

References

Amazon Web Services (2026). Amazon rekognition. <https://aws.amazon.com/rekognition/>. Accessed: 2026-04-26.

Google (2026). Google photos. <https://photos.google.com/>. Accessed: 2026-04-26.